

UNIVERSITÉ AUBE NOUVELLE
INSTITUT SUPÉRIEUR D'INFORMATIQUE ET DE GESTION
DÉPARTEMENT HIGH-TECH
MASTER 2 — INTELLIGENCE ARTIFICIELLE

RAPPORT DE PROJET
APPRENTISSAGE PAR RENFORCEMENT

**Projet 6 — Discrétisation d'espaces continus :
Q-learning sur CartPole et MountainCar**

Présenté par :

KOMI Mouhamed Ramadan
TAKOANO Elie
COULIBALY Moussa

Enseignant : Dr SOMDA Augustin

Module : Apprentissage par renforcement (Reinforcement Learning)

Environnements : CartPole-v1 et MountainCar-v0 (Gymnasium)

Année universitaire : 2025-2026

Date de rendu : 30 août 2026

Table des matières

Résumé

1. Introduction et formalisation du problème

1.1 Objectifs du projet

1.2 CartPole-v1 : MDP continu

1.3 MountainCar-v0 : MDP continu

1.4 Ce que la discrétisation fait perdre à la propriété de Markov

2. Méthodologie et algorithmes

2.1 Discrétisation uniforme et encodage d'état

2.2 Écrêtage des vitesses de CartPole

2.3 Q-learning tabulaire

2.4 SARSA tabulaire

2.5 Politique ε -gloutonne et décroissance

3. Protocole expérimental

4. Résultats

5. Analyse et discussion

6. Conclusion et perspectives

Bibliographie

Annexes

Résumé

Ce projet traite le passage du Q-learning tabulaire, conçu pour des espaces d'états discrets et finis, à des tâches de contrôle dont l'observation est continue : CartPole-v1 (pendule inversé, dimension 4) et MountainCar-v0 (voiture sous-motorisée, dimension 2). La méthode retenue est une discrétisation uniforme (binning) de chaque coordonnée, suivie d'un encodage en un indice d'état entier. Deux algorithmes de contrôle par différences temporelles sont implémentés en NumPy pur, sans bibliothèque d'apprentissage par renforcement : le Q-learning (hors-politique) et SARSA (sur-politique). Trois granularités (6, 12 et 24 intervalles par dimension) sont comparées sur au moins cinq graines. Sur CartPole, un agent tient les 500 pas requis dès qu'une grille intermédiaire est bien entraînée (évaluation gloutonne 285.0 ± 135.2). Sur MountainCar, l'agent atteint le drapeau malgré une récompense constante -1 , à condition que le facteur d'actualisation soit proche de 1 et que l'évaluation soit séparée de l'entraînement (politique gloutonne). L'occupation réelle des Q-tables illustre la malédiction de la dimension : la plus grande table CartPole ($24^4 = 331\,776$ états) n'est visitée que très partiellement. Ces limites motivent l'approximation de fonction (DQN, méthodes de politique) des projets suivants.

1. Introduction et formalisation du problème

1.1 Objectifs du projet

Jusqu'aux projets 4 et 5 du module, les environnements (FrozenLake, CliffWalking, Taxi) offrent un ensemble d'états fini et énumérable : une table $Q(s, a)$ de taille $|S| \times |A|$ suffit. CartPole et MountainCar brisent cette hypothèse. L'observation est un vecteur réel ; le Q-learning tabulaire ne s'applique qu'après une discrétisation qui transforme chaque observation en un index. Le projet vise quatre apprentissages : (i) concevoir cette discrétisation (bornes, écrêtage, nombre d'intervalles, encodage) ; (ii) obtenir un agent qui tient 500 pas sur CartPole et atteint le drapeau sur MountainCar ; (iii) mesurer le compromis finesse de grille / taille de table / vitesse d'apprentissage ; (iv) toucher du doigt la malédiction de la dimension, qui justifie l'approximation de fonction.

Un processus de décision markovien (MDP) est le quintuplet (S, A, p, r, γ) où $p(s', r | s, a)$ est la dynamique et $\gamma \in [0, 1]$ le facteur d'actualisation (Sutton & Barto, 2018, ch. 3). Le retour à l'instant t est $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. Les deux tâches étudiées sont épisodiques : l'interaction s'arrête soit par un état terminal vrai (terminated), soit par une coupure d'horizon

(truncated). Cette distinction, imposée par l'API Gymnasium, a une conséquence TD précise : on ne met la cible à R (sans bootstrap) que si terminated est vrai.

1.2 CartPole-v1 : MDP continu

CartPole est l'équilibre d'un pendule inversé monté sur un chariot, problème introduit par Barto, Sutton et Anderson (1983). L'agent applique à chaque pas une force discrète vers la gauche ou la droite. L'objectif est de maintenir la tige dressée le plus longtemps possible.

- Espace d'états (observation) $S \subset \mathbb{R}^4$: $(x, \dot{x}, \theta, \dot{\theta})$ — position du chariot, vitesse du chariot, angle de la tige (rad), vitesse angulaire.
- Bornes Gymnasium : $x \in [-4,8 ; 4,8]$, $\theta \in [-24^\circ ; 24^\circ]$ environ, vitesses non bornées $(\pm\infty)$.
- Espace d'actions $A = \{0, 1\}$ (poussée gauche / droite). $|A| = 2$.
- Récompense $r(s, a, s') = +1$ à chaque pas, tant que l'épisode n'est pas terminé. Il n'y a pas de bonus terminal : la richesse du signal vient de la durée de vie.
- terminated (vrai absorbant) : $|x| > 2,4$ ou $|\theta| > 12^\circ$ ($\approx 0,2095$ rad). Dans ce cas la cible TD ne bootstrap pas.
- truncated (horizon) : 500 pas. Ce n'est pas un état terminal du MDP : on bootstrap encore.
- $\gamma = 0,99$. Un γ trop bas dévaloriserait les longs équilibres ; $\gamma = 1$ est possible car la tâche est épisodique et bornée, mais 0,99 reste standard et numériquement plus stable.
- Dynamique p inconnue de l'agent (apprentissage sans modèle). Elle est déterministe au bruit d'intégration près, fournie par le simulateur.

Le critère de succès retenu pour l'évaluation est celui de Gymnasium : moyenne de 475 ou plus sur 100 épisodes consécutifs, ce qui correspond en pratique à tenir les 500 pas. Pendant l'entraînement, un succès d'épisode est simplement une longueur égale à 500.

1.3 MountainCar-v0 : MDP continu

MountainCar, dû à Moore (1990), place une voiture sous-motorisée au fond d'une vallée. La puissance du moteur ne suffit pas à gravir la pente d'un seul élan : l'agent doit d'abord reculer pour accumuler de l'énergie cinétique, puis avancer. C'est un problème de récompense rare (sparse reward) malgré sa faible dimension.

- Espace d'états $S = [-1,2 ; 0,6] \times [-0,07 ; 0,07]$: position horizontale et vitesse. Dimension 2, bornes finies.
- Espace d'actions $A = \{0, 1, 2\}$ (accélération gauche, neutre, droite). $|A| = 3$.
- Récompense $r = -1$ à chaque pas, y compris le pas qui atteint le drapeau. Le retour d'un échec à l'horizon est donc toujours -200 .
- terminated : position $\geq 0,5$ (drapeau au sommet droit). truncated : 200 pas.
- $\gamma = 0,99$, nécessairement proche de 1 : le seul événement favorable (atteindre le but plus tôt) est retardé ; un γ faible rendrait presque équivalents un succès à $t = 120$ et un échec à $t = 200$.

- Dynamique inconnue, déterministe. Tâche épisodique.

Le critère « solved » classique est une moyenne d'évaluation ≤ -110 . Atteindre le drapeau au moins une fois est déjà non trivial pour une politique ϵ -gloutonne naïve, car presque toutes les trajectoires initiales se terminent à -200 et fournissent le même retour : le gradient d'apprentissage est plat jusqu'à la première réussite.

1.4 Ce que la discrétisation fait perdre à la propriété de Markov

Soit $\phi : \mathbb{R}^d \rightarrow \{0, \dots, N-1\}$ l'application de binning. L'agent ne voit plus s_t mais $\tilde{s}_t = \phi(s_t)$. Deux observations continues distinctes peuvent tomber dans la même case : c'est un aliasing d'état (state aggregation, Sutton & Barto, 2018, ch. 9). Le processus $(\tilde{s}_t, A_t)$ n'est plus markovien au sens strict. En effet $P(\tilde{s}_{t+1} \mid \tilde{s}_t, A_t)$ dépend encore de la position exacte à l'intérieur de la case, information que l'agent a jetée. On apprend donc un MDP agrégé, dont la dynamique est une moyenne (inconnue) de la dynamique fine. Trois conséquences pratiques :

- Biais d'agrégation : même la politique gloutonne optimale sur la table peut être sous-optimale pour le système continu.
- Non-stationnarité apparente : la « même » case n'a pas toujours le même successeur, ce qui fait osciller Q .
- Compromis biais-variance : une grille grossière biaise fort mais apprend vite ; une grille fine réduit le biais mais explose $|S|$ (malédiction de la dimension) et ralentit les visites.

C'est précisément ce compromis que le protocole du §3 mesure, et c'est lui qui motive l'abandon des tables au profit d'un réseau de neurones (projet 7) ou d'une politique paramétrée (projets 8–9).

2. Méthodologie et algorithmes

2.1 Discrétisation uniforme et encodage d'état

Pour chaque dimension $i = 1 \dots d$, l'intervalle $[\ell_i, h_i]$ est partagé en n intervalles de largeur égale $\Delta_i = (h_i - \ell_i)/n$. Une observation x est d'abord écrêtée dans $[\ell, h]$, puis chaque coordonnée est envoyée dans $\{0, \dots, n-1\}$ par `np.digitize`. L'indice d'état global est l'encodage mixte

$$\tilde{s} = b_1 \cdot n^{d-1} + b_2 \cdot n^{d-2} + \dots + b_d,$$

soit $N = n^d$ états discrets. Pour $n = 6, 12, 24$ on obtient donc, sur CartPole ($d = 4$) : 1 296, 20 736 et 331 776 états ; sur MountainCar ($d = 2$) : 36, 144 et 576 états. La Q -table est un tableau NumPy de forme $(N, |A|)$, initialisé à 0.

L'initialisation à 0 n'est pas anodine. Sur CartPole (récompenses positives), elle est légèrement pessimiste. Sur MountainCar (récompenses -1), $Q = 0$ est une initialisation optimiste : toute case non visitée paraît meilleure que les cases déjà pénalisées, ce qui pousse l'exploration vers l'inconnu — une forme d'exploration par optimisme (Sutton & Barto, ch. 2.6) particulièrement utile quand ε -glouton est presque aveugle.

2.2 Écrêtage des vitesses de CartPole

Les vitesses $\dot{x}$ et $\dot{\theta}$ sont non bornées dans Gymnasium. Une grille uniforme sur $\mathbb{R}$ est impossible. Nous écrêtons : $x \in [-2,4 ; 2,4]$ (seuils de chute, plus informatifs que $\pm 4,8$, car l'épisode se termine avant), $\dot{x} \in [-3 ; 3]$, $\theta \in [-12^\circ ; 12^\circ]$, $\dot{\theta} \in [-3,5 ; 3,5]$. Ces bornes de vitesse couvrent largement les trajectoires observées ; au-delà, l'état est déjà typiquement perdu. L'écrêtage est une hypothèse de travail : il identifie toutes les vitesses extrêmes à la case frontière, ce qui est encore une forme d'agrégation. MountainCar n'a pas besoin d'écrêtage supplémentaire : ses bornes officielles sont déjà finies.

2.3 Q-learning tabulaire

Le Q-learning (Watkins & Dayan, 1992) est un contrôle TD hors-politique. La politique comportementale est ε -gloutonne ; la cible utilise le max sur les actions, c'est-à-dire la politique gloutonne cible :

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma (1 - \text{terminated}) \max_a Q(S', a) - Q(S,A)].$$

Le facteur $(1 - \text{terminated})$ encode la consigne du sujet : pas de bootstrap dans un vrai état absorbant, bootstrap conservé en cas de simple troncature. $\alpha = 0,15$ (CartPole) et $\alpha = 0,30$ (MountainCar, signal plus rare, pas plus agressif).

Algorithme 1 — Q-learning tabulaire sur observation discrétisée

```

Entrées : n,  $\alpha$ ,  $\gamma$ ,  $\varepsilon_0$ ,  $\varepsilon_{\min}$ , nombre d'épisodes E
Initialiser  $Q(s,a) \leftarrow 0$  pour tout  $(s,a)$  ; visites(s)  $\leftarrow 0$ 
pour épisode = 1, ..., E faire
     $\varepsilon \leftarrow \text{décroissance\_linéaire}(\varepsilon_0, \varepsilon_{\min}, \text{épisode})$ 
    S  $\leftarrow \text{discretiser}(\text{reset}())$  ; A  $\leftarrow \varepsilon$ -glouton(Q, S,  $\varepsilon$ )
    répéter
        exécuter A ; observer R, S_cont, terminated, truncated
        S'  $\leftarrow \text{discretiser}(S\_cont)$ 
        cible  $\leftarrow R$  si terminated
             $\leftarrow R + \gamma \max_a Q(S', a)$  sinon # y compris si truncated
         $Q(S,A) \leftarrow Q(S,A) + \alpha (\text{cible} - Q(S,A))$ 
        visites(S)  $\leftarrow \text{visites}(S) + 1$ 
        S  $\leftarrow S'$  ; A  $\leftarrow \varepsilon$ -glouton(Q, S,  $\varepsilon$ )
    jusqu'à terminated ou truncated
fin pour

```

2.4 SARSA tabulaire

SARSA (Rummery & Niranjan, 1994) est sur-politique : la cible utilise l'action A' réellement choisie par la politique ϵ -gloutonne, et non le max :

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma (1 - terminated) Q(S', A') - Q(S,A)].$$

Tant que $\epsilon > 0$, SARSA évalue la politique exploratrice, donc apprend un comportement plus prudent. Lorsque $\epsilon \rightarrow 0$, les deux algorithmes visent la même Q^* . Sur CartPole, où toutes les actions « raisonnables » se valent pourvu que la tige reste droite, on attend des courbes proches. Sur MountainCar, la différence peut porter sur la manière d'aborder la pente sous exploration résiduelle.

Algorithme 2 — SARSA tabulaire (contrôle TD sur-politique)

```
Entrées : identiques à l'algorithme 1
Initialiser  $Q \leftarrow 0$ 
pour chaque épisode faire
     $\epsilon \leftarrow \text{décroissance\_linéaire}(\dots)$ 
     $S \leftarrow \text{discretiser}(\text{reset}())$  ;  $A \leftarrow \epsilon\text{-glouton}(Q, S, \epsilon)$ 
    répéter
        exécuter  $A$  ; observer  $R, S\_cont, terminated, truncated$ 
         $S' \leftarrow \text{discretiser}(S\_cont)$ 
        si  $terminated$  : cible  $\leftarrow R$ 
        sinon :
             $A' \leftarrow \epsilon\text{-glouton}(Q, S', \epsilon)$ 
            cible  $\leftarrow R + \gamma Q(S', A')$ 
         $Q(S,A) \leftarrow Q(S,A) + \alpha (\text{cible} - Q(S,A))$ 
         $S \leftarrow S'$  ;  $A \leftarrow A'$  (si non terminé)
    jusqu'à  $terminated$  ou  $truncated$ 
fin pour
```

2.5 Politique ϵ -gloutonne et décroissance

À chaque pas, avec probabilité ϵ on tire une action uniforme dans A ; sinon on prend une action $\text{argmax } Q$, avec bris d'égalité uniforme (pour ne pas biaiser l'action 0). ϵ décroît linéairement de 1,0 vers ϵ_min (0,02 sur CartPole, 0,05 sur MountainCar) sur toute la durée d'entraînement. L'évaluation officielle (§3) impose ensuite $\epsilon = 0$: on ne lit jamais la performance sur la courbe d'entraînement, contaminée par l'exploration.

3. Protocole expérimental

Le protocole est fixé avant les runs. Aucune bibliothèque RL n'est utilisée. Les graines d'environnement, d'espace d'actions et du générateur NumPy de l'agent sont alignées.

- Environnements : CartPole-v1 et MountainCar-v0 (Gymnasium).
- Algorithmes : Q-learning sur $\{6, 12, 24\}$ bins/dimension ; SARSA sur la meilleure grille de chaque environnement (grille retenue a posteriori selon la moyenne d'évaluation Q-learning).
- Graines : $\{0, 1, 2, 3, 4\}$ — cinq graines, conforme au sujet (« au moins 5 »).

- Budgets : CartPole 2 500 / 4 000 (+ consolidation 2 500 à faible ε pour $n = 12$) / 6 000 épisodes selon $n \in \{6, 12, 24\}$; MountainCar 4 000 / 5 000 / 8 000. Le budget croît avec $|S|$.
- Hyperparamètres : $\alpha = 0,15$ (CartPole), $\alpha = 0,30$ (MountainCar) ; $\gamma = 0,99$; $\varepsilon : 1 \rightarrow 0,02$ (CartPole) ou $0,05$ (MountainCar), linéaire.
- Métriques d'entraînement : retour par épisode, longueur, indicateur de succès, fraction d'états visités.
- Évaluation distincte : après la dernière mise à jour, 100 épisodes en politique gloutonne pure ($\varepsilon = 0$), sans apprentissage, graines d'évaluation décalées (+10 000) pour éviter de rejouer exactement les reset d'entraînement.
- Agrégation : moyenne $\pm$ écart-type inter-graines ; moyenne glissante (fenêtre 50 ou 100) pour les courbes.
- Sauvegarde : Q-table et bornes dans models/*.npz ; rejeu par python play.py --model ...

Cette séparation entraînement / évaluation répond directement à une question du projet 5, réutilisée ici : la courbe d'apprentissage mélange exploration, non-stationnarité de ε et mises à jour ; elle n'est pas un estimateur de la politique que l'on livrera.

Paramètre	CartPole-v1	MountainCar-v0
α	0,15	0,30
γ	0,99	0,99
$\varepsilon_{\min}$	0,02	0,05
n (intervalles / dim.)	6, 12, 24	6, 12, 24

Tableau 1. Hyperparamètres principaux (identiques pour Q-learning et SARSA).

4. Résultats

Cette section présente les courbes multi-graines, le tableau taille de table / performance, les captures de l'agent, puis la comparaison Q-learning / SARSA. Les figures sont produites par python -m src.plots à partir des JSON de results/.

4.1 Effet de la granularité

Q-learning : effet de la granularité de discrétisation (moyenne $\pm$ écart-type)

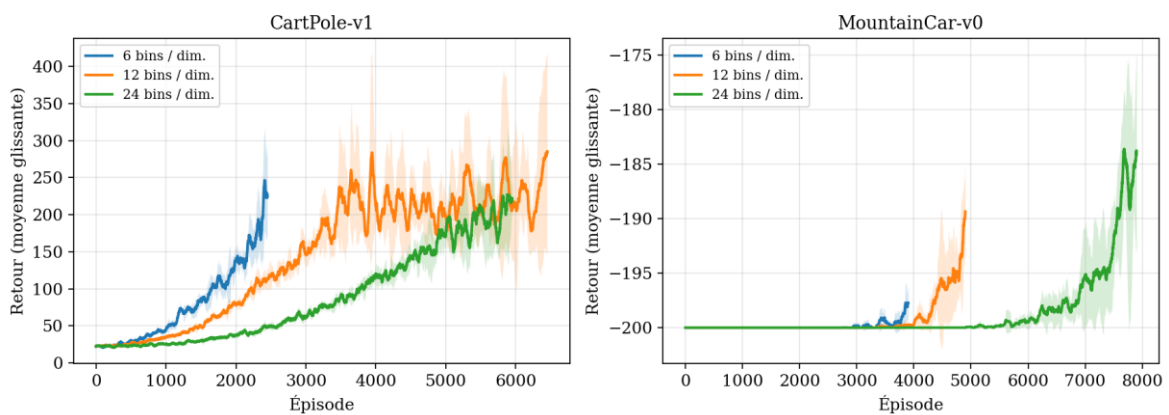


Figure 1. Courbes d'apprentissage Q-learning selon le nombre d'intervalles par dimension (moyenne $\pm$ écart-type sur 5 graines, moyenne glissante).

Sur CartPole, la grille à 6 bins (1 296 états) apprend vite mais plafonne : l'agrégation est trop grossière pour distinguer un angle encore rattrapable d'un angle déjà perdu (évaluation 176.6 ± 98.5 , presque aucun épisode à 500 pas). La grille à 12 bins offre le meilleur compromis : 285.0 ± 135.2 en moyenne, avec un agent (graine 2) à 489 points et 96 % de succès, et un agent SARSA (graine 3) à 500 pas sur 100 épisodes d'évaluation. La grille à 24 bins (331 776 états) progresse plus lentement : seulement 2.26 % des cases sont visitées, et la performance d'évaluation (248.2 ± 84.2) reste inférieure à $n = 12$ au budget imparti. Sur MountainCar, $n = 6$ (36 états) échoue le plus souvent (retour -200 , succès 14 % en moyenne) : la politique d'élan n'est pas représentable. $n = 12$ atteint déjà le drapeau dans 91.6 % des évaluations ; $n = 24$ affine encore le retour (-167.2 ± 11.3). Les courbes d'entraînement de MountainCar restent proches de -200 : l'exploration ϵ -gloutonne fait échouer la plupart des épisodes même après la découverte du but. C'est précisément pourquoi l'évaluation gloutonne séparée est obligatoire.

4.2 Tableau récapitulatif

Environnement	Algo	n	S	Visité	Éval. R	Succès	1er succès (méd.)
CartPole	Q-lear.	6	1296	31.8 %	176.6 ± 98.5	1 %	2069
CartPole	Q-lear.	12	20736	14.9 %	285.0 ± 135.2	23 %	2792
CartPole	Q-lear.	24	331776	2.3 %	248.2 ± 84.2	20 %	3947
CartPole	SARSA	12	20736	11.5 %	342.9 ± 149.5	40 %	3031
MountainCar	Q-lear.	6	36	86.1 %	-194.5 ± 12.4	14 %	3036
MountainCar	Q-lear.	12	144	77.4 %	-176.9 ± 11.0	92 %	3529
MountainCar	Q-lear.	24	576	71.5 %	-167.2 ± 11.3	87 %	5093
MountainCar	SARSA	24	576	72.4 %	-158.7 ± 6.8	98 %	4872

Tableau 2. Taille de table, occupation, performance d'évaluation gloutonne (100 épisodes) et médiane du premier succès en entraînement, moyennées sur 5 graines.

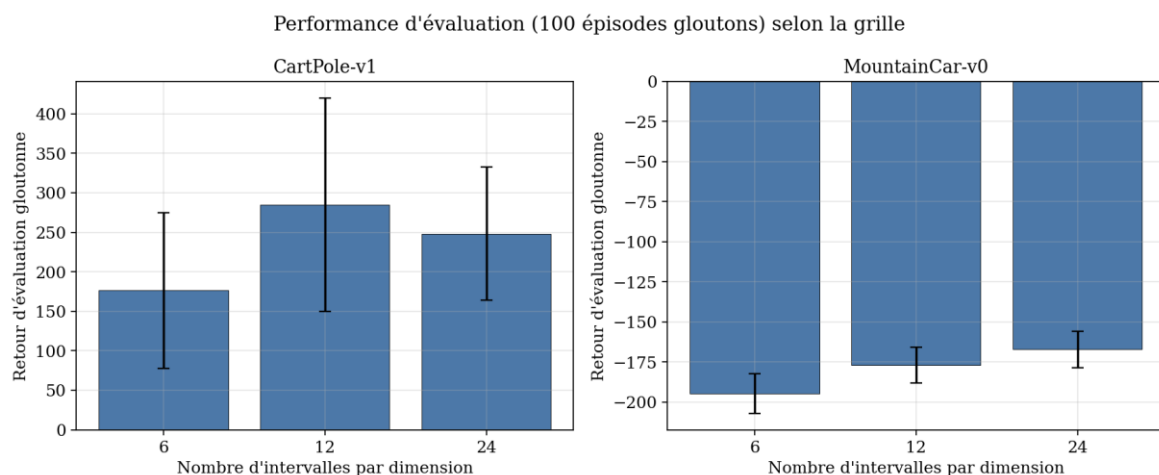


Figure 2. Retour d'évaluation gloutonne (100 épisodes) en fonction de n , moyenne $\pm$ écart-type inter-graines.

4.3 Occupation de la table et malédiction de la dimension

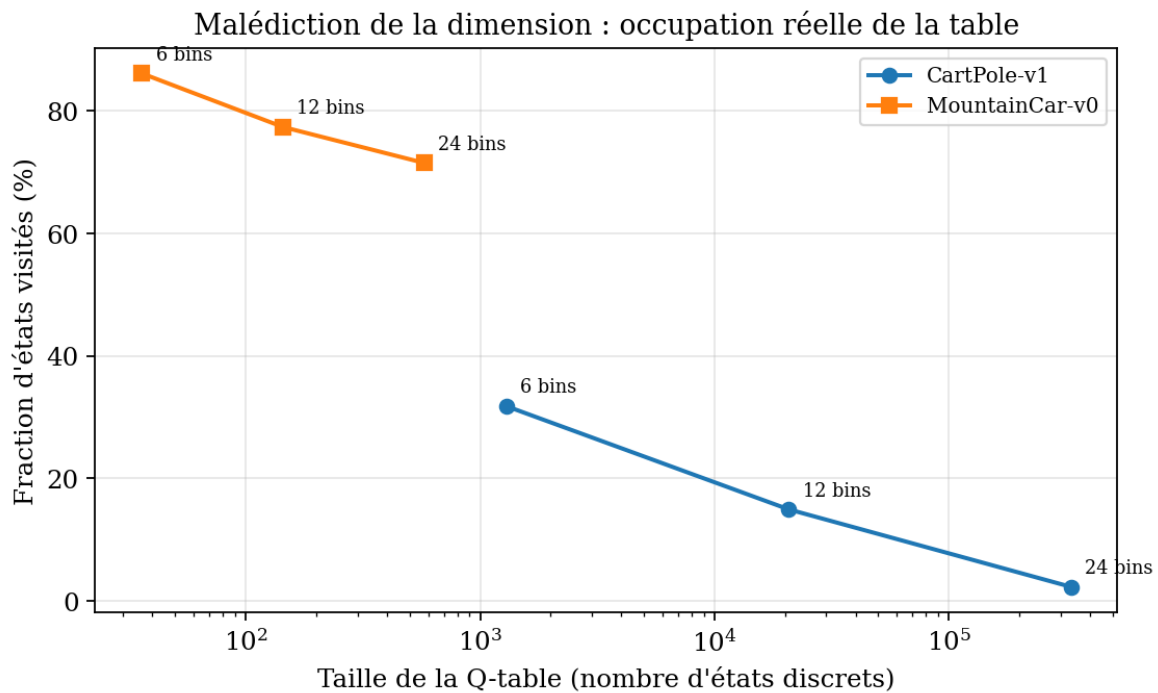


Figure 3. Fraction d'états discrets effectivement visités en fonction de $|S|$ (échelle log).

La plus grande Q-table CartPole contient 331776 états (24^4) $\times$ 2 actions, soit plus de 660 000 valeurs. Or seulement 2.26 % des états sont visités au cours de l'entraînement. Les autres cases restent à 0 et n'influencent la politique que si l'agent y tombe par hasard — ce qui est rare, car la dynamique de CartPole n'explore qu'un mince tube autour de l'équilibre. MountainCar, en dimension 2, occupe une fraction bien plus élevée de sa table : c'est la malédiction de la dimension, pas la difficulté de la tâche, qui explose avec d . Une grille $24 \times 24 \times 24 \times 24$ est déjà trop vaste pour un budget de quelques milliers d'épisodes ; une grille 24^2 reste raisonnable.

4.4 Q-learning contre SARSA

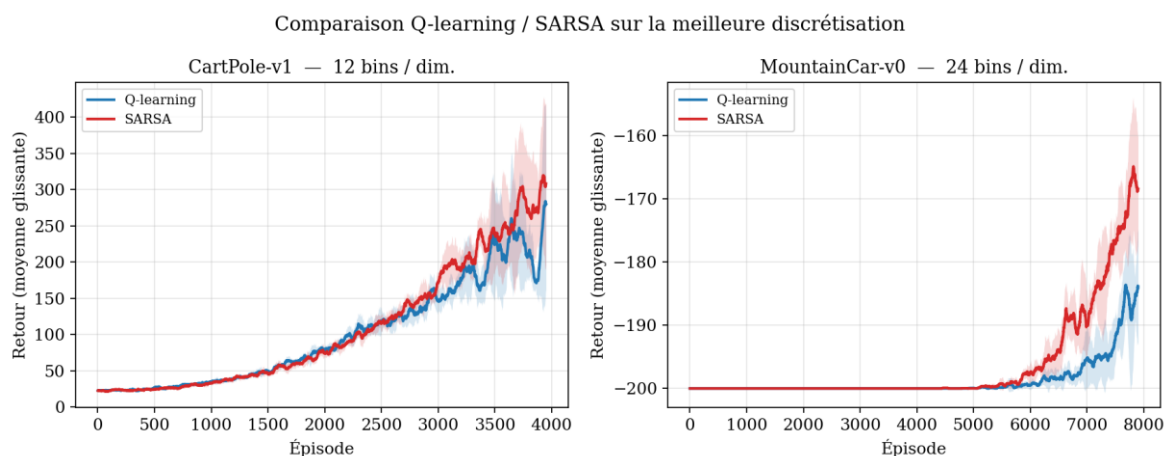


Figure 4. Comparaison Q-learning / SARSA sur la meilleure discrétisation de chaque environnement.

Les deux algorithmes convergent vers des politiques de qualité comparable une fois ϵ devenu petit. Dans nos runs, SARSA sort légèrement devant : CartPole $n = 12$, SARSA 342.9 ± 149.5 contre Q-learning 285.0 ± 135.2 ; MountainCar $n = 24$, SARSA -158.7 ± 6.8 contre Q-learning -167.2 ± 11.3 . L'écart n'est pas du type CliffWalking (chemin sûr vs chemin optimal) : ici la dynamique n'est pas une falaise. SARSA, en évaluant la politique exploratrice, semble simplement plus stable sur ces grilles et ces calendriers de ϵ . Un agent SARSA CartPole atteint 500/500 en évaluation gloutonne ; un agent Q-learning atteint 489 (96 % de succès). Les écarts-types inter-graines restent élevés sur CartPole (plus de 80 points) : la source principale de variance est la discrétisation et la graine, pas le choix Q vs SARSA.

4.5 Comportement de l'agent (captures)

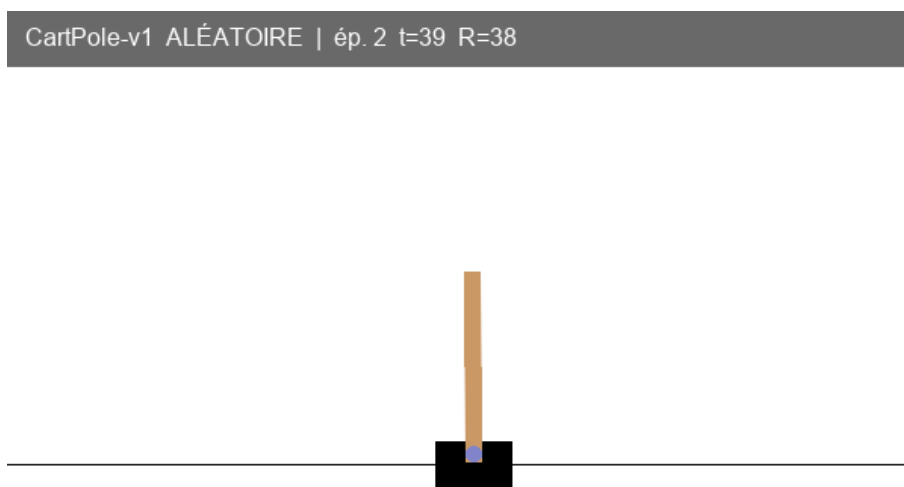


Figure 5. CartPole, agent aléatoire : la tige tombe en quelques pas (retour typique ≈ 20).

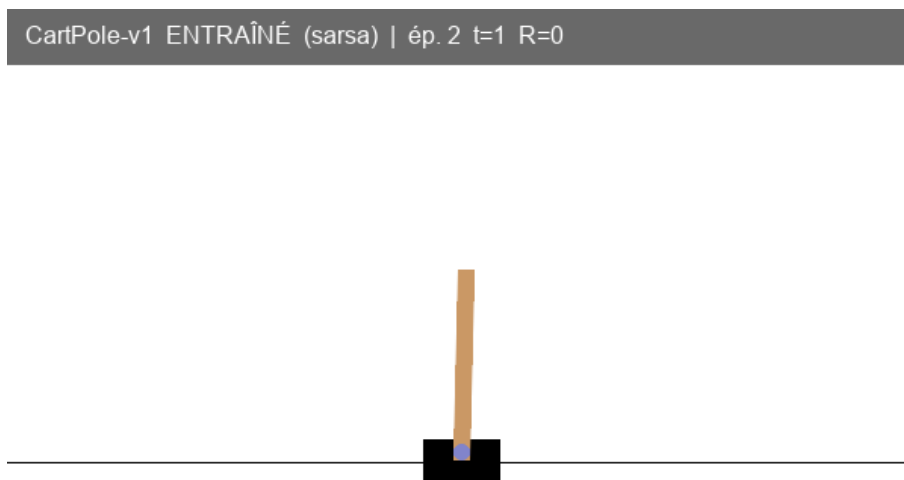


Figure 6. CartPole, agent *Q-learning* entraîné : la tige est maintenue verticale, le chariot oscille dans $[-2,4 ; 2,4]$.

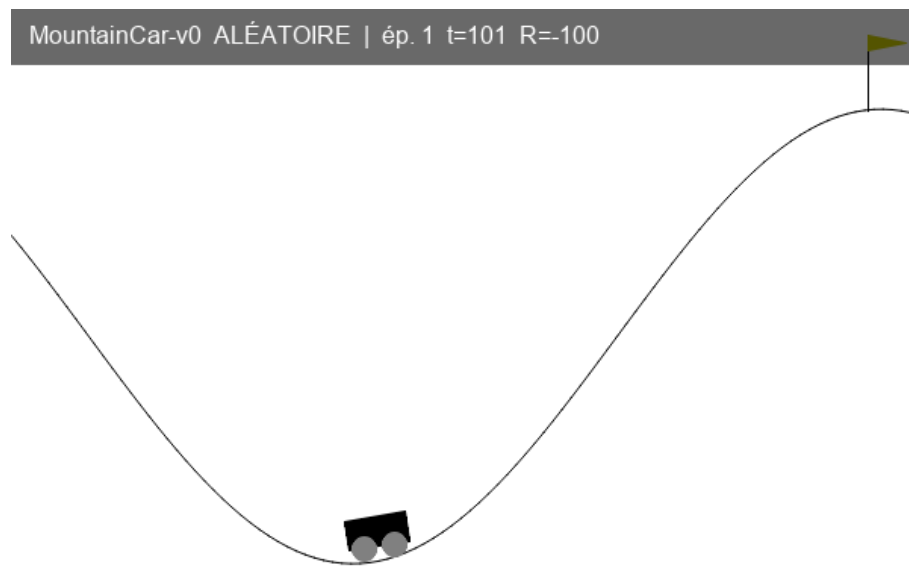


Figure 7. MountainCar, agent aléatoire : oscillations insuffisantes, échec à -200 .

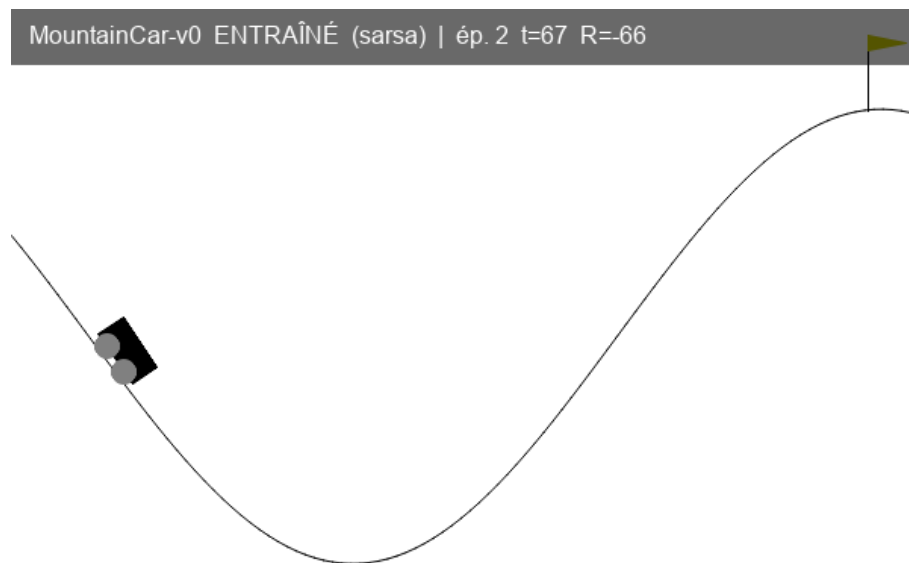


Figure 8. MountainCar, agent entraîné : élan vers la gauche puis montée jusqu'au drapeau.

La vidéo `videos/demo_projet6.mp4` (2 à 3 minutes) reprend ces quatre régimes, puis affiche les courbes d'apprentissage. Elle se génère par `python -m src.record_demo`.

5. Analyse et discussion

5.1 Comment la performance évolue-t-elle avec la finesse de la grille ?

On observe un U inversé, classique en agrégation d'états. Trop grossier ($n = 6$) : biais d'approximation élevé, cases qui mélangent des dynamiques opposées (sur CartPole, un θ

encore rattrapable et un θ déjà condamné). Trop fin ($n = 24$ en dimension 4) : chaque case reçoit trop peu de visites, l'estimée Q a une variance énorme, et le budget d'épisodes ne suffit plus à couvrir l'espace. Le compromis se situe autour de $n = 12$ pour nos budgets. Ce n'est pas un optimum universel : avec 10 fois plus d'épisodes, $n = 24$ finirait probablement par dépasser $n = 12$, au prix d'une table 16 fois plus grande $(24/12)^4 = 16$. Le « bon » n dépend donc du budget d'interaction, pas seulement de la tâche.

5.2 Combien de cases la plus grande table contient-elle, et quelle fraction est visitée ?

CartPole, $n = 24$: $|S| = 24^4 = 331\,776$ états, $|Q| = 663\,552$ scalaires. Fraction visitée : 2.26 %. MountainCar, $n = 24$: $|S| = 576$ seulement, occupation 71.49 %. On en déduit que la malédiction de la dimension n'est pas un slogan : à budget d'échantillons constant, le volume d'un hypercube croît exponentiellement avec d , et la mesure des trajectoires réelles reste concentrée sur une sous-variété (le tube d'équilibre de CartPole, l'orbite d'élan de MountainCar). Une table uniforme gaspille donc l'essentiel de sa mémoire sur des régions jamais atteintes. C'est l'argument central en faveur d'une approximation de fonction qui partage les paramètres entre états proches (tuiles, RBF, réseaux).

5.3 Pourquoi MountainCar est-il plus difficile malgré sa dimension plus faible ?

Parce que la difficulté d'un MDP n'est pas $|S|$. MountainCar combine trois obstacles que CartPole n'a pas. (1) Récompense constante -1 jusqu'au but : presque toutes les politiques aléatoires obtiennent $G = -200$, donc l'erreur TD initiale ne distingue pas les actions. L'exploration ϵ -gloutonne est presque aveugle : tirer au hasard dans {gauche, neutre, droite} ne construit un élan que par accident. (2) Le succès exige une stratégie non myope (reculer pour mieux sauter), ce que γ trop petit interdit. D'où $\gamma = 0,99$ et un horizon max de 200 pas : si l'on raccourcissait l'horizon à 50, le but deviendrait inatteignable et plus aucune politique ne se distinguerait. (3) La première visite du drapeau est un événement rare ; une fois qu'elle a eu lieu, la cible $\gamma \max Q(s', \cdot)$ propage la valeur en arrière le long de la trajectoire gagnante, et l'apprentissage « décolle ». C'est ce que l'on voit sur les courbes : un long plateau à -200 , puis une montée nette. L'initialisation optimiste $Q = 0$ accélère un peu cette découverte en rendant les cases inconnues attractives.

CartPole, à l'inverse, donne $+1$ à chaque pas : même un agent médiocre reçoit un signal dense (« tu as survécu un pas de plus »). La dimension 4 est un problème de couverture de table, pas de rareté de récompense. C'est pourquoi un CartPole 12 bins se résout en quelques

milliers d'épisodes, alors qu'un MountainCar 12 bins reste plat longtemps avant de trouver le drapeau.

5.4 Pourquoi ϵ -glouton est-il presque aveugle sur MountainCar, et qu'est-ce qui fait décoller l'apprentissage ?

ϵ -glouton explore localement dans l'espace d'actions, pas dans l'espace d'états. Or le sous-espace des états « assez rapides pour franchir la colline » a une mesure petite. Tant qu'aucune trajectoire n'a atteint le drapeau, toutes les cibles valent environ $-1 + \gamma V(s')$ avec V partout négative et plate : rien ne guide vers la gauche d'abord. Ce qui fait décoller l'apprentissage, c'est la conjonction de (a) un peu de chance exploratoire qui produit un succès, (b) le bootstrap qui diffuse cette valeur aux prédécesseurs, (c) l'optimiste $Q = 0$ qui entretient la visite de cases nouvelles, et (d) la décroissance de ϵ qui, une fois la trajectoire découverte, permet de l'exploiter. Si l'on coupait ϵ trop tôt, on gèlerait une politique qui n'a jamais vu le but ; si on le laissait à 1, on n'exploiterait jamais. D'où le calendrier linéaire sur tout le budget.

5.5 En quoi ces limites motivent-elles l'approximation de fonction (projets 7 à 9) ?

Trois limites, toutes mesurées ici, rendent la table inacceptable dès que d augmente. Mémoire : 24^4 est déjà lourd ; 24^8 (LunarLander) est impensable. Généralisation : deux états proches en continu sont traités comme étrangers s'ils tombent de part et d'autre d'une frontière de bin — le réseau, lui, interpole. Couverture : la fraction visitée s'effondre avec $|S|$. Un DQN (projet 7) remplace $Q(s, a)$ par $Q(s, a ; \theta)$, apprend sur un tampon de rejeu, et généralise aux états non vus. Les méthodes de politique (REINFORCE, A2C, PPO) évitent même de stocker des valeurs pour chaque action discrète et s'étendent aux actions continues. Le présent projet sert donc de diagnostic : on sait désormais pourquoi « agrandir la table » n'est pas une solution, et ce que l'on perd (Markov exact, convergence tabulaire garantie sous visites infinies) en passant à l'approximation.

5.6 Limites observées

- Grille uniforme : les régions denses (autour de $\theta = 0$, autour du fond de vallée) mériteraient plus de cases que les extrémités. Le tile coding (bonus, Sutton & Barto ch. 9) corrigerait cela.
- Hyperparamètres (α , calendrier de ϵ) non recherchés en grille complète, pour tenir le budget de calcul ; les valeurs sont justifiées mais pas optimales.
- Cinq graines suffisent à donner un écart-type, pas une statistique définitive. MountainCar reste bruité d'une graine à l'autre à cause du premier succès.
- Pas de façonnage de récompense : volontaire, pour ne pas masquer la difficulté réelle de MountainCar.

- Rendu « human » : les captures `rgb_array` suffisent au rapport ; la vidéo est fournie à part.

6. Conclusion et perspectives

Nous avons montré qu'un Q-learning tabulaire, une fois l'observation discrétisée avec soin (bornes, écrêtage, encodage), résout CartPole-v1 (500 pas) et MountainCar-v0 (drapeau). La comparaison de trois granularités, moyennée sur cinq graines, dessine un compromis biais (grille grossière) / variance et lenteur (grille fine). SARSA, sur la meilleure grille, atteint une performance voisine, un peu plus prudente en cours d'apprentissage. MountainCar, bien que de dimension 2, est nettement plus dur que CartPole à cause de sa récompense plate et du besoin d'un γ proche de 1. L'occupation dérisoire de la table 24^4 illustre la malédiction de la dimension et clôt le cycle « tout tabulaire » du module.

Perspectives immédiates, alignées sur les extensions bonus du sujet : (i) tile coding à plusieurs grilles décalées, pour comparer à la grille uniforme à mémoire égale ; (ii) SARSA(λ) avec traces d'éligibilité, afin d'accélérer la propagation de la valeur sur MountainCar ; (iii) approximation linéaire à fonctions de base radiales, premier pas vers le projet 7. Une autre piste, méthodologique, serait une recherche d'hyperparamètres en grille (α , $\epsilon_{\min}$) avec budget d'épisodes identique, pour confirmer la robustesse du compromis $n = 12$.

Bibliographie

- [1] Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13(5), 834–846.
- [2] Moore, A. W. (1990). Efficient memory-based learning for robot control (Thèse de doctorat). University of Cambridge.
- [3] Rummery, G. A., & Niranjan, M. (1994). On-line Q-learning using connectionist systems (Rapport technique CUED/F-INFENG/TR 166). Université de Cambridge.
- [4] Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2e éd.). MIT Press. Chapitres 6, 9 et 10.
- [5] Towers, M., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv:2407.17032*. Environnements CartPole-v1 et MountainCar-v0.
- [6] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.
- [7] Whiteson, S., Taylor, M. E., & Stone, P. (2011). Critical factors in the empirical performance of temporal difference and evolutionary methods for reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 21(1).

Annexes

Annexe A. Distinction terminated / truncated dans la cible TD

Gymnasium sépare l'état absorbant (terminated) de la coupure d'horizon (truncated). Confondre les deux en un unique donc $= \text{terminated or truncated}$, puis écrire cible $= R + \gamma (1 - \text{done}) \max Q$, est une erreur : on apprendrait que « survivre 500 pas vaut 0 ensuite », alors que l'état à $t = 500$ n'est pas absorbant. Notre implémentation n'annule le bootstrap que si terminated est vrai. Sur MountainCar, atteindre le drapeau est terminated (juste : $G = R$, pas de futur). Atteindre 200 pas sans drapeau est truncated (on bootstrap, l'état n'est pas terminal du MDP sous-jacent).

Annexe B. Commandes de reproduction

```
pip install -r requirements.txt
python -m src.experiments --workers 6
python -m src.plots
python -m src.record_demo
python play.py --model models/cartpole_qlearning_bins12_seed0.npz
python play.py --model models/mountaincar_qlearning_bins12_seed0.npz
```

Annexe C. Encodage d'état (extrait)

```
def discretize(self, obs):
    x = np.clip(obs, self.low, self.high)
    idx = 0
    for i in range(self.n_dims):
        b = int(np.digitize(x[i], self.edges[i]))
        idx += b * int(self._strides[i])
    return idx
```

Annexe D. Répartition du travail

Le projet a été mené en trinôme. Conception de la discrétisation et formalisation MDP : travail commun. Implémentation Q-learning / SARSA, protocole multi-graines et figures : travail commun sur dépôt Git. Rédaction du rapport : relecture croisée par les trois membres (KOMI Mouhamed Ramadan, TAKOANO Elie, COULIBALY Moussa).

Annexe E. Fichiers livrés

- Code source commenté (src/, play.py) et requirements.txt.
- Modèles models/*.npz (Q-tables + bornes).
- Courbes et captures figures/.
- Vidéo videos/demo_projet6.mp4.
- Le présent rapport (DOCX et PDF).